

Complete JavaScript & React Interview Guide

Table of Contents

1. Event Loop & Asynchronous JavaScript
 2. JavaScript Fundamentals
 3. Arrays & Objects
 4. CSS & Styling
 5. React Fundamentals
 6. State Management
 7. Advanced JavaScript Concepts
 8. Company-Specific Questions
-

Event Loop & Asynchronous JavaScript

1. What is precedence in the Event Loop?

Answer: The Event Loop follows a specific priority order:

1. **Call Stack** (highest priority)
2. **Microtask Queue** (Promises, queueMicrotask)
3. **Macrotask Queue** (setTimeout, setInterval, DOM events)

javascript

```
console.log('1'); // Call stack - executes first
```

```
setTimeout(() => console.log('2'), 0); // Macrotask queue
```

```
Promise.resolve().then(() => console.log('3')); // Microtask queue
```

```
console.log('4'); // Call stack
```

```
// Output: 1, 4, 3, 2
```

2. What is the difference between setTimeout and setInterval?

setTimeout: Executes a function once after a delay **setInterval:** Executes a function repeatedly at intervals

javascript

// setTimeout - runs once after 1 second

```
setTimeout(() => {  
  console.log('Executed once');  
}, 1000);
```

// setInterval - runs every 1 second

```
const intervalId = setInterval(() => {  
  console.log('Executed repeatedly');  
}, 1000);
```

// Clear interval

```
clearInterval(intervalId);
```

3. Where do you use the Rest Operator?

Answer: Rest operator (`(...)`) collects multiple elements into an array.

javascript

// Function parameters

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b, 0);  
}  
sum(1, 2, 3, 4); // 10
```

// Array destructuring

```
const [first, ...rest] = [1, 2, 3, 4];  
console.log(first); // 1  
console.log(rest); // [2, 3, 4]
```

// Object destructuring

```
const {name, ...otherProps} = {name: 'John', age: 30, city: 'NY'};  
console.log(otherProps); // {age: 30, city: 'NY'}
```

JavaScript Fundamentals

4. What is "hoisting" in JavaScript?

Answer: Hoisting moves variable and function declarations to the top of their scope.

during the compilation phase, before the code is executed

javascript

// This works due to hoisting

```
console.log(x); // undefined (not error)
```

```
var x = 5;
```

// Function declarations are fully hoisted

```
sayHello(); // "Hello!" - works
```

```
function sayHello() {  
  console.log("Hello!");  
}
```

// let and const are hoisted but not initialized

```
console.log(y); // ReferenceError
```

```
let y = 10;
```

5. What are the differences between `let`, `var`, and `const`?

Feature	var	let	const
Scope	Function	Block	Block
Hoisting	Yes, initialized with undefined	Yes, but not initialized	Yes, but not initialized
Re-declaration	Allowed	Not allowed	Not allowed
Re-assignment	Allowed	Allowed	Not allowed

javascript

```
// var - function scoped
function example() {
  if (true) {
    var x = 1;
  }
  console.log(x); // 1 - accessible outside block
}

// let - block scoped
function example2() {
  if (true) {
    let y = 1;
  }
  console.log(y); // ReferenceError - not accessible
}

// const - block scoped, cannot be reassigned
const z = 1;
z = 2; // TypeError
```

6. What is the difference between `==` and `===`?

`==`: Loose equality (type coercion) `===`: Strict equality (no type coercion)

javascript

```
console.log(5 == '5'); // true (string converted to number)
console.log(5 === '5'); // false (different types)

console.log(null == undefined); // true
console.log(null === undefined); // false

console.log(0 == false); // true
console.log(0 === false); // false
```

7. Explain how `this` works in JavaScript

Answer: `this` refers to the object that is executing the current function.

javascript

// Global context

```
console.log(this); // Window (browser) or global (Node.js)
```

// Object method

```
const person = {  
  name: 'John',  
  greet() {  
    console.log(this.name); // 'John'  
  }  
};
```

// Arrow functions don't have their own 'this'

```
const person2 = {  
  name: 'Jane',  
  greet: () => {  
    console.log(this.name); // undefined (inherits from parent scope)  
  }  
};
```

// Constructor function

```
function Person(name) {  
  this.name = name;  
}  
  
const john = new Person('John');
```

8. What's the difference between `.call()` and `.apply()`?

Answer: Both invoke functions with a specific `this` context, but differ in how arguments are passed.

javascript

```
function greet(greeting, punctuation) {  
  console.log(greeting + ' ' + this.name + punctuation);  
}
```

```
const person = { name: 'John' };
```

```
// call() - arguments passed individually  
greet.call(person, 'Hello', '!'); // "Hello John!"
```

```
// apply() - arguments passed as array  
greet.apply(person, ['Hello', '!']); // "Hello John!"
```

```
// bind() - returns new function with bound context  
const boundGreet = greet.bind(person);  
boundGreet('Hello', '!'); // "Hello John!"
```

Arrays & Objects

9. Have you heard of `array.reverse()`?

Answer: `reverse()` reverses an array in place and returns the reference.

javascript

```
const arr = [1, 2, 3, 4, 5];  
console.log(arr.reverse()); // [5, 4, 3, 2, 1]  
console.log(arr); // [5, 4, 3, 2, 1] - original array is modified
```

```
// Non-mutating alternative  
const original = [1, 2, 3, 4, 5];  
const reversed = [...original].reverse();  
console.log(original); // [1, 2, 3, 4, 5] - unchanged  
console.log(reversed); // [5, 4, 3, 2, 1]
```

10. What is meant by Shallow copy and Deep copy?

Shallow Copy: Copies only the first level of properties **Deep Copy:** Copies all levels of nested properties

javascript

// Shallow copy

```
const original = { a: 1, b: { c: 2 } };  
const shallow = { ...original };  
shallow.b.c = 3;  
console.log(original.b.c); // 3 - original affected!
```

// Deep copy

```
const deep = JSON.parse(JSON.stringify(original));  
deep.b.c = 4;  
console.log(original.b.c); // 3 - original not affected
```

// Custom deep copy function

```
function deepCopy(obj) {  
  if (obj === null || typeof obj !== 'object') return obj;  
  if (obj instanceof Date) return new Date(obj);  
  if (obj instanceof Array) return obj.map(item => deepCopy(item));  
  
  const copied = {};  
  Object.keys(obj).forEach(key => {  
    copied[key] = deepCopy(obj[key]);  
  });  
  return copied;  
}
```

11. Have you used the reduce function in JS?

Answer: `reduce()` executes a reducer function on each array element, resulting in a single output value.

javascript

// Basic sum

```
const numbers = [1, 2, 3, 4, 5];
const sum = numbers.reduce((accumulator, current) => accumulator + current, 0);
console.log(sum); // 15
```

// Find maximum

```
const max = numbers.reduce((max, current) => current > max ? current : max);
```

// Group by property

```
const people = [
  { name: 'John', age: 30 },
  { name: 'Jane', age: 25 },
  { name: 'Bob', age: 30 }
];

const groupedByAge = people.reduce((groups, person) => {
  const age = person.age;
  if (!groups[age]) groups[age] = [];
  groups[age].push(person);
  return groups;
}, {});
```

12. What is the difference between map and reduce?

Feature	map	reduce
Purpose	Transform each element	Accumulate to single value
Return	New array (same length)	Single value
Use case	Data transformation	Aggregation/calculation

javascript

```
const numbers = [1, 2, 3, 4, 5];
```

// map - transforms each element

```
const doubled = numbers.map(x => x * 2);
console.log(doubled); // [2, 4, 6, 8, 10]
```

// reduce - accumulates to single value

```
const sum = numbers.reduce((acc, curr) => acc + curr, 0);
console.log(sum); // 15
```

13. What parameters does the map function accept?

Answer: `map()` callback receives three parameters:

javascript

```
const array = ['a', 'b', 'c'];
```

```
array.map((element, index, originalArray) => {  
  console.log(`Element: ${element}, Index: ${index}, Array: ${originalArray}`);  
  return element.toUpperCase();  
});
```

// Output:

// Element: a, Index: 0, Array: a,b,c

// Element: b, Index: 1, Array: a,b,c

// Element: c, Index: 2, Array: a,b,c

Promises & Async Programming

14. What is the difference between a Promise and a Callback?

Callback: Function passed as argument to another function **Promise:** Object representing eventual completion of an asynchronous operation

Callbacks are functions passed as arguments to other functions, to be executed after an asynchronous operation completes. Promises, on the other hand, are objects representing the eventual result of an asynchronous operation, offering a more structured and cleaner approach, especially for handling multiple asynchronous operations and error management

javascript

// Callback example

```
function fetchDataCallback(callback) {  
  setTimeout(() => {  
    callback(null, 'Data received');  
  }, 1000);  
}
```

```
fetchDataCallback((error, data) => {  
  if (error) console.error(error);  
  else console.log(data);  
});
```

// Promise example

```
function fetchDataPromise() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve('Data received');  
    }, 1000);  
  });  
}
```

```
fetchDataPromise()  
  .then(data => console.log(data))  
  .catch(error => console.error(error));
```

// Async/await

```
async function fetchData() {  
  try {  
    const data = await fetchDataPromise();  
    console.log(data);  
  } catch (error) {  
    console.error(error);  
  }  
}
```

CSS & Styling

15. What position attributes in CSS have you used?

Answer: CSS position property values:

CSS

/ static - default, follows normal document flow */*

```
.static { position: static; }
```

/ relative - positioned relative to its normal position */*

```
.relative {  
  position: relative;  
  top: 10px;  
  left: 20px;  
}
```

/ absolute - positioned relative to nearest positioned ancestor */*

```
.absolute {  
  position: absolute;  
  top: 50px;  
  right: 0;  
}
```

/ fixed - positioned relative to viewport */*

```
.fixed {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 100%;  
}
```

/ sticky - switches between relative and fixed */*

```
.sticky {  
  position: sticky;  
  top: 0;  
}
```

16. What is Flexbox?

Answer: Flexbox is a CSS layout method for arranging items in a container.

```
.flex-container {
  display: flex;
  justify-content: center; /* horizontal alignment */
  align-items: center; /* vertical alignment */
  flex-direction: row; /* or column */
  flex-wrap: wrap; /* or nowrap */
  gap: 10px;
}

.flex-item {
  flex: 1; /* grow, shrink, basis */
  flex-grow: 1;
  flex-shrink: 0;
  flex-basis: auto;
}
```

17. What is the difference between display=none and visibility=hidden?

Property	Space occupied	Accessible to screen readers	Events
display: none	No	No	No
visibility: hidden	Yes	No	No

```
.hidden-display {
  display: none; /* Completely removed from layout */
}

.hidden-visibility {
  visibility: hidden; /* Hidden but space preserved */
}
```

React Fundamentals

18. What Hooks have you used?

Answer: Common React Hooks:

functions that let you “hook into” React state and lifecycle features from function components

javascript

```
import React, {
  useState,
  useEffect,
  useContext,
  useReducer,
  useCallback,
  useMemo,
  useRef
} from 'react';

function MyComponent() {
  // useState - manage state
  const [count, setCount] = useState(0);

  // useEffect - side effects
  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]);

  // useCallback - memoize function
  const increment = useCallback(() => {
    setCount(prev => prev + 1);
  }, []);

  // useMemo - memoize value
  const expensiveValue = useMemo(() => {
    return count * 2;
  }, [count]);

  // useRef - reference DOM element
  const inputRef = useRef(null);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>+</button>
      <input ref={inputRef} />
    </div>
  );
}
```

19. What is the purpose of useCallback?

Answer: `useCallback` memoizes functions to prevent unnecessary re-renders.

javascript

```
import React, { useState, useCallback } from 'react';

function Parent() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState("");

  // Without useCallback - function recreated on every render
  const handleClick = () => {
    setCount(count + 1);
  };

  // With useCallback - function memoized
  const handleClickMemo = useCallback(() => {
    setCount(prev => prev + 1);
  }, []); // Empty dependency array

  return (
    <div>
      <Child onClick={handleClickMemo} />
      <input value={name} onChange={(e) => setName(e.target.value)} />
    </div>
  );
}

const Child = React.memo(({ onClick }) => {
  console.log('Child rendered');
  return <button onClick={onClick}>Click me</button>;
});
```

20. What are Class-based Lifecycle methods?

Answer: Class components have lifecycle methods for different phases:

javascript

```
class MyComponent extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { count: 0 };  
  }  
  
  // Mounting  
  componentDidMount() {  
    console.log('Component mounted');  
    // API calls, subscriptions  
  }  
  
  // Updating  
  componentDidUpdate(prevProps, prevState) {  
    if (prevState.count !== this.state.count) {  
      console.log('Count updated');  
    }  
  }  
  
  // Unmounting  
  componentWillUnmount() {  
    console.log('Component will unmount');  
    // Cleanup: remove listeners, cancel requests  
  }  
  
  render() {  
    return <div>Count: {this.state.count}</div>;  
  }  
}
```

21. How can you achieve lifecycle methods in functional components?

Answer: Use `useEffect` hook:

javascript

```
import React, { useState, useEffect } from 'react';
```

```
function MyComponent() {  
  const [count, setCount] = useState(0);
```

```
  // componentDidMount  
  useEffect(() => {  
    console.log('Component mounted');  
    // API calls, subscriptions  
  }, []); // Empty dependency array
```

```
  // componentDidUpdate  
  useEffect(() => {  
    console.log('Count updated');  
  }, [count]); // Depends on count
```

```
  // componentWillUnmount  
  useEffect(() => {  
    return () => {  
      console.log('Component will unmount');  
      // Cleanup  
    };  
  }, []);
```

```
  return <div>Count: {count}</div>;  
}
```

22. What are Pure Components?

Answer: Pure Components only re-render when props or state change (shallow comparison).

javascript

// Class component

```
class PureComponent extends React.PureComponent {  
  render() {  
    return <div>{this.props.name}</div>;  
  }  
}
```

// Functional component with React.memo

```
const PureFunctional = React.memo(function MyComponent({ name }) {  
  return <div>{name}</div>;  
});
```

// Custom comparison

```
const PureCustom = React.memo(function MyComponent({ user }) {  
  return <div>{user.name}</div>;  
}, (prevProps, nextProps) => {  
  return prevProps.user.id === nextProps.user.id;  
});
```

23. What are Higher Order Components (HOCs)?

Answer: HOCs are functions that take a component and return a new component.

Higher-Order Components (HOCs) are a powerful pattern in React for reusing component logic. They are functions that take a component as an argument and return a new, enhanced component. This pattern allows you to share common functionality (like data fetching, authentication, or styling) across multiple components without repeating code

javascript

// HOC example

```
function withLogger(WrappedComponent) {  
  return function LoggerComponent(props) {  
    useEffect(() => {  
      console.log('Component rendered:', WrappedComponent.name);  
    });  
  
    return <WrappedComponent {...props} />;  
  };  
}
```

// Usage

```
const MyComponent = ({ name }) => <div>{name}</div>;  
const MyComponentWithLogger = withLogger(MyComponent);
```

// Authentication HOC

```
function withAuth(WrappedComponent) {  
  return function AuthComponent(props) {  
    const [isAuthenticated, setIsAuthenticated] = useState(false);  
  
    if (!isAuthenticated) {  
      return <div>Please login</div>;  
    }  
  
    return <WrappedComponent {...props} />;  
  };  
}
```

24. Have you used the Context API?

Answer: Context API provides a way to pass data through component tree without prop drilling.

javascript

```
import React, { createContext, useContext, useState } from 'react';
```

```
// Create context
```

```
const ThemeContext = createContext();
```

```
// Provider component
```

```
function ThemeProvider({ children }) {
```

```
  const [theme, setTheme] = useState('light');
```

```
  return (
```

```
    <ThemeContext.Provider value={{ theme, setTheme }}>
```

```
      {children}
```

```
    </ThemeContext.Provider>
```

```
  );
```

```
}
```

```
// Consumer component
```

```
function ThemedButton() {
```

```
  const { theme, setTheme } = useContext(ThemeContext);
```

```
  return (
```

```
    <button
```

```
      style={{ backgroundColor: theme === 'light' ? '#fff' : '#333' }}
```

```
      onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}
```

```
    >
```

```
      Toggle Theme
```

```
    </button>
```

```
  );
```

```
}
```

```
// App component
```

```
function App() {
```

```
  return (
```

```
    <ThemeProvider>
```

```
      <ThemedButton />
```

```
    </ThemeProvider>
```

```
  );
```

```
}
```

State Management

25. Why use Redux when React has state management?

Answer: Redux provides centralized state management for complex applications.

React State Issues:

- Prop drilling
- State scattered across components
- Difficult to debug
- No time-travel debugging

Redux Benefits:

- Single source of truth
- Predictable state updates
- DevTools for debugging
- Middleware support

javascript

// Redux store

```
import { createStore } from 'redux';
```

```
const initialState = { count: 0 };
```

```
function counterReducer(state = initialState, action) {  
  switch (action.type) {  
    case 'INCREMENT':  
      return { count: state.count + 1 };  
    case 'DECREMENT':  
      return { count: state.count - 1 };  
    default:  
      return state;  
  }  
}
```

```
const store = createStore(counterReducer);
```

// React component

```
import { useSelector, useDispatch } from 'react-redux';
```

```
function Counter() {  
  const count = useSelector(state => state.count);  
  const dispatch = useDispatch();  
  
  return (  
    <div>  
      <p>Count: {count}</p>  
      <button onClick={() => dispatch({ type: 'INCREMENT' })}>+</button>  
      <button onClick={() => dispatch({ type: 'DECREMENT' })}>-</button>  
    </div>  
  );  
}
```

26. How does Redux work?

Answer: Redux follows a unidirectional data flow:

1. **Action** → 2. **Reducer** → 3. **Store** → 4. **View**

javascript

// 1. Action

```
const increment = () => ({ type: 'INCREMENT' });
```

// 2. Reducer

```
const counterReducer = (state = { count: 0 }, action) => {  
  switch (action.type) {  
    case 'INCREMENT':  
      return { count: state.count + 1 };  
    default:  
      return state;  
  }  
};
```

// 3. Store

```
const store = createStore(counterReducer);
```

// 4. View subscribes to store

```
store.subscribe(() => {  
  console.log(store.getState());  
});
```

// Dispatch action

```
store.dispatch(increment());
```

27. What are Redux Middlewares?

Answer: Middleware provides extension points between dispatching an action and reaching the reducer.

javascript

```
// Redux Thunk - for async actions
const asyncAction = () => {
  return async (dispatch, getState) => {
    const response = await fetch('/api/data');
    const data = await response.json();
    dispatch({ type: 'SET_DATA', payload: data });
  };
};

// Logger middleware
const loggerMiddleware = store => next => action => {
  console.log('Dispatching:', action);
  const result = next(action);
  console.log('Next state:', store.getState());
  return result;
};

// Apply middleware
const store = createStore(
  rootReducer,
  applyMiddleware(thunk, loggerMiddleware)
);
```

Advanced JavaScript Concepts

28. What is a closure?

Answer: A closure is a function that has access to variables from its outer scope even after the outer function has returned.

javascript

```
function outerFunction(x) {  
  return function innerFunction(y) {  
    return x + y; // innerFunction has access to x  
  };  
}
```

```
const add5 = outerFunction(5);  
console.log(add5(3)); // 8
```

// Practical example - data privacy

```
function createCounter() {  
  let count = 0;  
  
  return {  
    increment: () => ++count,  
    decrement: () => --count,  
    getCount: () => count  
  };  
}
```

```
const counter = createCounter();  
console.log(counter.getCount()); // 0  
counter.increment();  
console.log(counter.getCount()); // 1
```

29. What is prototypal inheritance?

Answer: JavaScript uses prototypal inheritance where objects inherit directly from other objects.

through something called prototype chain

```
let animal = {  
  eats: true,  
  walk() {  
    console.log("Animal walks.");  
  }  
};  
let rabbit = {  
  jumps: true,  
  __proto__: animal // rabbit inherits from animal  
};  
console.log(rabbit.eats); // true (inherited from animal)  
rabbit.walk(); // Animal walks. (inherited from animal)
```


javascript

```
// Constructor function
function Person(name) {
  this.name = name;
}

Person.prototype.greet = function() {
  return `Hello, I'm ${this.name}`;
};

function Student(name, grade) {
  Person.call(this, name);
  this.grade = grade;
}

// Set up inheritance
Student.prototype = Object.create(Person.prototype);
Student.prototype.constructor = Student;

Student.prototype.study = function() {
  return `${this.name} is studying`;
};

const student = new Student('John', 'A');
console.log(student.greet()); // "Hello, I'm John"
console.log(student.study()); // "John is studying"
```

30. What are different ways to create objects?

Answer: Multiple ways to create objects in JavaScript:

javascript

// 1. Object literal

```
const obj1 = { name: 'John', age: 30 };
```

// 2. Object constructor

```
const obj2 = new Object();
```

```
obj2.name = 'John';
```

```
obj2.age = 30;
```

// 3. Constructor function

```
function Person(name, age) {
```

```
  this.name = name;
```

```
  this.age = age;
```

```
}
```

```
const obj3 = new Person('John', 30);
```

// 4. Object.create()

```
const personPrototype = {
```

```
  greet() { return `Hello, I'm ${this.name}`; }
```

```
};
```

```
const obj4 = Object.create(personPrototype);
```

```
obj4.name = 'John';
```

// 5. ES6 Classes

```
class PersonClass {
```

```
  constructor(name, age) {
```

```
    this.name = name;
```

```
    this.age = age;
```

```
  }
```

```
}
```

```
const obj5 = new PersonClass('John', 30);
```

// 6. Factory function

```
function createPerson(name, age) {
```

```
  return { name, age };
```

```
}
```

```
const obj6 = createPerson('John', 30);
```

Company-Specific Questions

31. Common React Performance Optimization Questions

Q: How do you optimize React application performance?

javascript

// 1. Use React.memo for component memoization

```
const ExpensiveComponent = React.memo(({ data }) => {  
  return <div>{data.name}</div>;  
});
```

// 2. Use useMemo for expensive calculations

```
function Component({ items }) {  
  const expensiveValue = useMemo(() => {  
    return items.reduce((sum, item) => sum + item.value, 0);  
  }, [items]);  
  
  return <div>{expensiveValue}</div>;  
}
```

// 3. Use useCallback for function memoization

```
const Parent = () => {  
  const [count, setCount] = useState(0);  
  
  const handleClick = useCallback(() => {  
    setCount(prev => prev + 1);  
  }, []);  
  
  return <Child onClick={handleClick} />;  
};
```

// 4. Code splitting with React.lazy

```
const LazyComponent = React.lazy(() => import('./LazyComponent'));  
  
function App() {  
  return (  
    <Suspense fallback=<div>Loading...</div>>  
      <LazyComponent />  
    </Suspense>  
  );  
}
```

32. Error Handling in React

Q: How do you handle errors in React?

// Error Boundary class component

```
class ErrorBoundary extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { hasError: false };  
  }  
  
  static getDerivedStateFromError(error) {  
    return { hasError: true };  
  }  
  
  componentDidCatch(error, errorInfo) {  
    console.error('Error caught:', error, errorInfo);  
  }  
  
  render() {  
    if (this.state.hasError) {  
      return <h1>Something went wrong.</h1>;  
    }  
    return this.props.children;  
  }  
}
```

// Usage

```
function App() {  
  return (  
    <ErrorBoundary>  
      <MyComponent />  
    </ErrorBoundary>  
  );  
}
```

// Try-catch with async operations

```
async function fetchData() {  
  try {  
    const response = await fetch('/api/data');  
    if (!response.ok) {  
      throw new Error('Network response was not ok');  
    }  
    return await response.json();  
  } catch (error) {  
    console.error('Fetch error:', error);  
    throw error;  
  }  
}
```

33. Custom Hooks

Q: How do you create custom hooks?

// Custom hook for API calls

```
function useApi(url) {  
  const [data, setData] = useState(null);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(null);
```

```
  useEffect(() => {  
    const fetchData = async () => {  
      try {  
        setLoading(true);  
        const response = await fetch(url);  
        const result = await response.json();  
        setData(result);  
      } catch (err) {  
        setError(err);  
      } finally {  
        setLoading(false);  
      }  
    };  
  });
```

```
  fetchData();  
  }, [url]);
```

```
  return { data, loading, error };  
}
```

// Usage

```
function MyComponent() {  
  const { data, loading, error } = useApi('/api/users');  
  
  if (loading) return <div>Loading...</div>;  
  if (error) return <div>Error: {error.message}</div>;  
  
  return <div>{data.map(user => <div key={user.id}>{user.name}</div>)}</div>;  
}
```

// Custom hook for localStorage

```
function useLocalStorage(key, initialValue) {  
  const [storedValue, setStoredValue] = useState(() => {  
    try {  
      const item = window.localStorage.getItem(key);  
      return item ? JSON.parse(item) : initialValue;  
    } catch (error) {  
      return initialValue;  
    }  
  });  
}
```



```
const setValue = (value) => {  
  try {  
    setStoredValue(value);  
    window.localStorage.setItem(key, JSON.stringify(value));  
  } catch (error) {  
    console.error('Error setting localStorage:', error);  
  }  
};  
  
return [storedValue, setValue];  
}
```

34. Testing Questions

Q: How do you test React components?

javascript

```
// Using React Testing Library
import { render, screen, fireEvent } from '@testing-library/react';
import Counter from './Counter';

test('counter increments when button is clicked', () => {
  render(<Counter />);

  const button = screen.getByText('Increment');
  const counter = screen.getByText('Count: 0');

  fireEvent.click(button);

  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});

// Testing hooks
import { renderHook, act } from '@testing-library/react';
import { useCounter } from './useCounter';

test('useCounter hook', () => {
  const { result } = renderHook(() => useCounter());

  expect(result.current.count).toBe(0);

  act(() => {
    result.current.increment();
  });

  expect(result.current.count).toBe(1);
});
```

35. Security Questions

Q: How do you handle security in React applications?

javascript

```
// 1. Sanitize user input
import DOMPurify from 'dompurify';

function SafeComponent({ userInput }) {
  const sanitizedInput = DOMPurify.sanitize(userInput);

  return <div dangerouslySetInnerHTML={{ __html: sanitizedInput }} />;
}

// 2. Validate props with PropTypes
import PropTypes from 'prop-types';

function UserProfile({ user }) {
  return <div>{user.name}</div>;
}

UserProfile.propTypes = {
  user: PropTypes.shape({
    name: PropTypes.string.isRequired,
    email: PropTypes.string.isRequired
  }).isRequired
};

// 3. Environment variables for sensitive data
// Use process.env for API keys, never hardcode
const API_KEY = process.env.REACT_APP_API_KEY;

// 4. Content Security Policy headers
// Set in server configuration or meta tags
```

Browser & DOM Questions

36. What is the difference between cookie, sessionStorage, and localStorage?

Feature	Cookie	sessionStorage	localStorage
Storage limit	4KB	5-10MB	5-10MB
Sent to server	Yes	No	No
Expiration	Set by expires/max-age	Tab close	Manual removal
Scope	Domain/path	Tab	Domain

javascript

// Cookie

```
document.cookie = "username=john; expires=Thu, 18 Dec 2024 12:00:00 UTC; path="/;
```

// sessionStorage

```
sessionStorage.setItem('sessionData', 'temporary');
```

```
const sessionData = sessionStorage.getItem('sessionData');
```

// localStorage

```
localStorage.setItem('userData', JSON.stringify({ name: 'John' }));
```

```
const userData = JSON.parse(localStorage.getItem('userData'));
```

37. What is the difference between `<script>`, `<script async>`, and `<script defer>`?

html

<!-- Regular script - blocks HTML parsing -->

```
<script src="script.js"> </script>
```

<!-- Async - downloads in parallel, executes immediately when ready -->

```
<script src="script.js" async> </script>
```

<!-- Defer - downloads in parallel, executes after HTML parsing -->

```
<script src="script.js" defer> </script>
```

Execution Order:

- Regular: Download → Execute → Continue parsing
- Async: Download in parallel → Execute when ready (may interrupt parsing)
- Defer: Download in parallel → Execute after parsing complete

38. Explain event delegation

Answer: Event delegation uses event bubbling to handle events on parent elements instead of individual child elements.

Event delegation is a technique where you attach a single event listener to a parent element, and that listener can handle events for current and future child elements using event bubbling.

How It Works (with Event Bubbling):

When an event (like click) is triggered on a child element...

It bubbles up to the parent (and up the DOM tree).

The parent's event listener can detect which child triggered the event using `event.target`.

javascript

// Instead of adding listeners to each button

```
document.querySelectorAll('button').forEach(button => {  
  button.addEventListener('click', handleClick);  
});
```

// Use event delegation on parent

```
document.getElementById('container').addEventListener('click', (e) => {  
  if (e.target.tagName === 'BUTTON') {  
    handleClick(e);  
  }  
});
```

// React example

```
function ButtonContainer() {  
  const handleClick = (e) => {  
    if (e.target.dataset.action) {  
      console.log('Action:', e.target.dataset.action);  
    }  
  };  
  
  return (  
    <div onClick={handleClick}>  
      <button data-action="save">Save</button>  
      <button data-action="cancel">Cancel</button>  
      <button data-action="delete">Delete</button>  
    </div>  
  );  
}
```

39. What is event bubbling and capturing?

Answer: Event propagation has three phases:

1. **Capturing Phase:** Event travels from root to target
2. **Target Phase:** Event reaches the target element
3. **Bubbling Phase:** Event bubbles up from target to root

javascript

// Event bubbling (default)

```
element.addEventListener('click', handler);
```

// Event capturing

```
element.addEventListener('click', handler, true);
```

// Stop propagation

```
function handleClick(e) {  
  e.stopPropagation(); // Stops bubbling  
  e.preventDefault(); // Prevents default behavior  
}
```

// Example

```
document.getElementById('parent').addEventListener('click', () => {  
  console.log('Parent clicked');  
});
```

```
document.getElementById('child').addEventListener('click', (e) => {  
  console.log('Child clicked');  
  e.stopPropagation(); // Parent won't receive event  
});
```

Advanced JavaScript Concepts

40. What are iterators and generators?

Answer: Iterators provide a way to access collection elements sequentially. Generators are functions that can pause and resume execution.

javascript

```
// Iterator
const numbers = [1, 2, 3];
const iterator = numbers[Symbol.iterator]();

console.log(iterator.next()); // { value: 1, done: false }
console.log(iterator.next()); // { value: 2, done: false }
console.log(iterator.next()); // { value: 3, done: false }
console.log(iterator.next()); // { value: undefined, done: true }

// Generator
function* numberGenerator() {
  yield 1;
  yield 2;
  yield 3;
}

const gen = numberGenerator();
console.log(gen.next()); // { value: 1, done: false }
console.log(gen.next()); // { value: 2, done: false }

// Infinite generator
function* infiniteNumbers() {
  let i = 0;
  while (true) {
    yield i++;
  }
}

const infinite = infiniteNumbers();
console.log(infinite.next()); // { value: 0, done: false }
console.log(infinite.next()); // { value: 1, done: false }
```

41. What is the difference between Map and Object?

Feature	Object	Map
Keys	Strings/Symbols	Any type
Size	Manual counting	.size property
Iteration	for...in, Object.keys()	for...of, .forEach()
Prototype	Has default keys	No default keys

javascript

// Object

```
const obj = {  
  name: 'John',  
  age: 30  
};
```

// Map

```
const map = new Map();  
map.set('name', 'John');  
map.set('age', 30);  
map.set(1, 'number key');  
map.set({}, 'object key');
```

```
console.log(map.size); // 4  
console.log(map.get('name')); // 'John'  
console.log(map.has('age')); // true
```

// Iteration

```
for (const [key, value] of map) {  
  console.log(key, value);  
}
```

42. What are Symbols used for?

Answer: Symbols create unique identifiers that won't collide with other properties.

javascript

// Creating symbols

```
const sym1 = Symbol('description');  
const sym2 = Symbol('description');  
console.log(sym1 === sym2); // false - each symbol is unique
```

// Using symbols as object keys

```
const obj = {  
  [sym1]: 'value1',  
  [sym2]: 'value2'  
};
```

// Well-known symbols

```
const customIterator = {  
  [Symbol.iterator]() {  
    let count = 0;  
    return {  
      next() {  
        if (count < 3) {  
          return { value: count++, done: false };  
        }  
        return { done: true };  
      }  
    };  
  }  
};
```

```
for (const value of customIterator) {  
  console.log(value); // 0, 1, 2  
}
```

43. What are Proxies used for?

Answer: Proxies intercept and customize operations on objects (property access, assignment, enumeration, etc.).

javascript

```
const target = { name: 'John', age: 30 };

const proxy = new Proxy(target, {
  get(target, prop) {
    console.log(`Getting ${prop}`);
    return target[prop];
  },

  set(target, prop, value) {
    console.log(`Setting ${prop} to ${value}`);
    if (prop === 'age' && typeof value !== 'number') {
      throw new Error('Age must be a number');
    }
    target[prop] = value;
    return true;
  }
});
```

```
console.log(proxy.name); // "Getting name", "John"
proxy.age = 31; // "Setting age to 31"
```

// Validation proxy

```
function createValidatedObject(validations) {
  return new Proxy({}, {
    set(target, prop, value) {
      if (validations[prop] && !validations[prop](value)) {
        throw new Error(`Invalid value for ${prop}`);
      }
      target[prop] = value;
      return true;
    }
  });
}
```

```
const user = createValidatedObject({
  email: value => value.includes('@'),
  age: value => value > 0 && value < 150
});
```

Modern JavaScript Features

44. What are template literals and tagged templates?

javascript

// Template literals

```
const name = 'John';
const age = 30;
const message = `Hello, my name is ${name} and I'm ${age} years old`;
```

// Multi-line strings

```
const multiline = `
  This is a
  multi-line
  string
`;
```

// Tagged templates

```
function highlight(strings, ...values) {
  return strings.reduce((result, string, i) => {
    return result + string + (values[i] ? `<mark>${values[i]}</mark>` : '');
  }, "");
}

const highlighted = highlight`Hello ${'World'}! Welcome to ${'JavaScript'}`;
// "Hello <mark>World</mark>! Welcome to <mark>JavaScript</mark>"
```

45. What is optional chaining and nullish coalescing?

Optional chaining (?.) allows safe navigation through nested objects, short-circuiting and returning undefined if any part of the chain is null or undefined. Nullish coalescing (??) provides a way to specify a default value when a variable is null or undefined.

javascript

// Optional chaining (?.)

```
const user = {  
  name: 'John',  
  address: {  
    street: '123 Main St'  
  }  
};
```

// Safe property access

```
console.log(user?.address?.street); // "123 Main St"  
console.log(user?.address?.zipCode); // undefined (no error)
```

// Safe method calls

```
user?.someMethod?.(); // Only calls if method exists
```

// Safe array access

```
const users = [{ name: 'John' }];  
console.log(users?.[0]?.name); // "John"
```

// Nullish coalescing (??)

```
const defaultValue = null ?? 'default'; // "default"  
const value = 0 ?? 'default'; // 0 (not "default")  
const value2 = "" ?? 'default'; // "" (not "default")
```

// Logical nullish assignment (??=)

```
let config = {};  
config.timeout ??= 5000; // Sets only if undefined or null
```

46. What are private class fields?

javascript

```
class BankAccount {  
  // Private fields (start with #)  
  #balance = 0;  
  #accountNumber;  
  
  // Private methods  
  #validateAmount(amount) {  
    return amount > 0;  
  }  
  
  constructor(accountNumber) {  
    this.#accountNumber = accountNumber;  
  }  
  
  deposit(amount) {  
    if (this.#validateAmount(amount)) {  
      this.#balance += amount;  
    }  
  }  
  
  getBalance() {  
    return this.#balance;  
  }  
}  
  
const account = new BankAccount('123456');  
account.deposit(100);  
console.log(account.getBalance()); // 100  
  
// console.log(account.#balance); // SyntaxError: Private field '#balance' must be declared in an enclosing class
```

Performance & Optimization

47. What are Web Workers?

Answer: Web Workers run JavaScript in background threads, allowing CPU-intensive tasks without blocking the main thread.

javascript

// main.js

```
const worker = new Worker('worker.js');
```

```
worker.postMessage({ numbers: [1, 2, 3, 4, 5] });
```

```
worker.onmessage = function(e) {  
  console.log('Result:', e.data);  
};
```

// worker.js

```
self.onmessage = function(e) {  
  const { numbers } = e.data;  
  const sum = numbers.reduce((acc, num) => acc + num, 0);  
  self.postMessage(sum);  
};
```

// React with Web Workers

```
function useWorker(workerFunction) {  
  const [result, setResult] = useState(null);  
  const [loading, setLoading] = useState(false);  
  
  const runWorker = useCallback((data) => {  
    setLoading(true);  
    const worker = new Worker(  
      URL.createObjectURL(new Blob(['${workerFunction}()']))  
    );  
  
    worker.postMessage(data);  
    worker.onmessage = (e) => {  
      setResult(e.data);  
      setLoading(false);  
      worker.terminate();  
    };  
  }, [workerFunction]);  
  
  return { result, loading, runWorker };  
}
```

48. How does JavaScript garbage collection work?

Answer: JavaScript uses automatic garbage collection with mark-and-sweep algorithm.

javascript

// Memory leaks to avoid

// 1. Global variables

```
var globalVar = 'This stays in memory';
```

// 2. Event listeners not removed

```
function addListener() {  
  const element = document.getElementById('button');  
  element.addEventListener('click', handler);  
  // Should remove: element.removeEventListener('click', handler);  
}
```

// 3. Closures holding references

```
function createClosure() {  
  const largeData = new Array(1000000).fill('data');  
  
  return function() {  
    // This closure keeps largeData in memory  
    console.log('Hello');  
  };  
}
```

// 4. Timers not cleared

```
const timer = setInterval(() => {  
  console.log('Running');  
}, 1000);  
// Should clear: clearInterval(timer);
```

// Good practices

```
function cleanupExample() {  
  const element = document.getElementById('button');  
  const handler = () => console.log('clicked');  
  
  element.addEventListener('click', handler);  
  
  // Cleanup function  
  return () => {  
    element.removeEventListener('click', handler);  
  };  
}
```

Testing & Debugging

49. What tools do you use for debugging JavaScript?

Answer: Various debugging tools and techniques:

// 1. Console methods

```
console.log('Basic logging');
console.error('Error message');
console.warn('Warning message');
console.table([{ name: 'John', age: 30 }]);
console.group('Group');
console.log('Inside group');
console.groupEnd();
```

// 2. Debugging with breakpoints

```
function debugExample() {
  const data = fetchData();
  debugger; // Browser will pause here
  return processData(data);
}
```

// 3. Performance monitoring

```
console.time('operation');
// Some operation
console.timeEnd('operation');
```

// 4. Error handling

```
try {
  riskyOperation();
} catch (error) {
  console.error('Error details:', {
    message: error.message,
    stack: error.stack,
    name: error.name
  });
}
```

// 5. Custom error classes

```
class CustomError extends Error {
  constructor(message, code) {
    super(message);
    this.name = 'CustomError';
    this.code = code;
  }
}
```

// 6. React debugging

```
function DebugComponent({ data }) {
  useEffect(() => {
    console.log('Component rendered with data:', data);
  });
}
```

```
return <div>{JSON.stringify(data, null, 2)}</div>;  
}
```

Final Tips for Interview Success

50. Common Coding Challenges

Debounce Implementation:

javascript

```
function debounce(func, delay) {  
  let timeoutId;  
  return function(...args) {  
    clearTimeout(timeoutId);  
    timeoutId = setTimeout(() => func.apply(this, args), delay);  
  };  
}
```

Throttle Implementation:

javascript

```
function throttle(func, limit) {  
  let inThrottle;  
  return function(...args) {  
    if (!inThrottle) {  
      func.apply(this, args);  
      inThrottle = true;  
      setTimeout(() => inThrottle = false, limit);  
    }  
  };  
}
```

Deep Clone:

javascript

```
function deepClone(obj) {  
  if (obj === null || typeof obj !== 'object') return obj;  
  if (obj instanceof Date) return new Date(obj);  
  if (obj instanceof Array) return obj.map(item => deepClone(item));  
  
  const cloned = {};  
  Object.keys(obj).forEach(key => {  
    cloned[key] = deepClone(obj[key]);  
  });  
  return cloned;  
}
```

Flatten Array:

javascript

```
function flatten(arr) {  
  return arr.reduce((flat, item) => {  
    return flat.concat(Array.isArray(item) ? flatten(item) : item);  
  }, []);  
}
```

Quick Reference Cheat Sheet

Array Methods

- `map()` - Transform elements
- `filter()` - Filter elements
- `reduce()` - Reduce to single value
- `find()` - Find first match
- `some()` - Test if any match
- `every()` - Test if all match

React Hooks

- `useState` - State management
- `useEffect` - Side effects
- `useCallback` - Memoize functions
- `useMemo` - Memoize values
- `useRef` - DOM references

- `useContext` - Context API

Performance Tips

- Use `React.memo` for component memoization
- Implement proper key props in lists
- Avoid inline functions in render
- Use code splitting with `React.lazy`
- Optimize bundle size with tree shaking

Common Patterns

- Higher-Order Components (HOCs)
- Render Props
- Custom Hooks
- Context API for state management
- Error Boundaries for error handling

Interview Preparation Strategy

1. **Practice coding problems** on platforms like LeetCode, HackerRank
2. **Build projects** using the technologies you'll be interviewed for
3. **Review fundamentals** regularly
4. **Practice explaining concepts** out loud
5. **Mock interviews** with peers or mentors
6. **Stay updated** with latest JavaScript/React features
7. **Prepare questions** to ask the interviewer

Good luck with your interview! 🚀

This guide covers the most commonly asked JavaScript and React interview questions. Practice these concepts regularly and you'll be well-prepared for your next interview.